ENGINEERING SPECIFICATION · REBUILD

Modular-Start

A complete, implementation-ready specification for rebuilding the platform from zero — data model, API surface, AI prompt construction, the credits economy, and page-by-page behaviour — paired with a new minimal, modern, glass visual language to replace the hand-drawn one.

Document Rebuild specification, v1.0

Source system `clubdeltomate/website-test` — React 19 + tRPC v11 + Drizzle/Postgres

Scope Full platform. Detailed page specs for Feed, Repos, Slides, Gallery, Users, Card and Credits.

Design brief Same product, new skin: minimal, modern, glass, and materially more intuitive.

Date 3 August 2026

CONTENTS

What is in this document

> Nine parts. Parts I–V are the machine: what the product is, how it is put together, what every endpoint does, how the AI prompts are built, and how credits move. Part VI is the seven page specifications you asked for. Parts VII–IX are the new design language and the plan to build it.

PART I — ORIENTATION

1	How to use this document	and the master build prompt
2	What the product is	the loop, in one page

PART II — ARCHITECTURE

3	Stack and repository layout	one deployable, three trees
4	Request lifecycle, auth and roles	JWT, three procedure tiers
5	Data model	25 tables, column by column
6	Migration strategy	why it is additive-only

PART III — THE API

7	Procedure conventions	
8	Full endpoint catalogue	21 routers, ~140 procedures

PART IV — THE AI LAYER

9	Providers and key resolution	BYOK, rotation, diversity
10	The deck engine prompt	how the big one is assembled
11	The other five prompts	path, coach, carousel, mathboard, grading
12	Output repair and validation	never lose a paid generation

PART V — THE CREDITS ECONOMY

13	Coins, tickets and the ledger	
14	The pricing formula	
15	Earn, spend, transfer — the full matrix	

PART VI — PAGE SPECIFICATIONS

16	Global shell and navigation	
17	Feed	/ and /feed
18	Repos	/repos and /repos/:slug
19	Slides	/slides and the player
20	Gallery	/gallery
21	Users	/users and /users/:id
22	Card	/card
23	Credits	/credits — new, replaces the Settings money tab

PART VII — THE NEW DESIGN SYSTEM

24 Principles

25 Tokens — colour, type, space, radius, glass

26 Components

27 Motion and accessibility

PART VIII — MAKING IT INTUITIVE

28 Diagnosis: what is confusing today

29 The fixes, surface by surface

PART IX — DELIVERY

30 Build order

31 Acceptance criteria

32 Environment and deployment

PART I · CHAPTER 1

How to use this document

This is written to be handed to an engineer — human or AI — who has never seen the original codebase and has to produce a working replacement. Everything they need is here; nothing requires reading the old source.

Three rules govern the rebuild:

1. **Behaviour is specified, design is not inherited.** Parts II–VI describe what the system *does*. Where they mention a visual detail, that detail is describing an *affordance*, not a look. The look comes from Part VII, which is new.
2. **Every AI action costs credits, for everybody.** There is no role that generates for free. Admin, moderator and ordinary user all pay from the same balance. This is a hard invariant — see Part V.
3. **The AI must never charge for nothing.** A generation that fails, returns unparseable output, or produces a deck below the quality floor is either repaired or refunded. Never both charged and lost.

1.1 The master build prompt

If you are driving this with an AI engineer, this is the framing prompt. Attach this document alongside it.

```
You are building Modular-Start from scratch. The attached specification is
authoritative for behaviour; you own the implementation.

PRODUCT
  An AI learning-and-presentation platform. A subject becomes a repository of
  units and lessons; each lesson is generated as a slide deck that teaches and
  quizzes; finished plays are logged back so later lessons build on earlier
  ones instead of repeating them. The same loop also runs restaurant menus,
  service catalogues, shop collections, walkthroughs and news briefings —
  only the labels change.

STACK (non-negotiable)
  React 19 + TypeScript + Vite . tRPC v11 (superjson) + TanStack Query v5
  Hono on Node . Drizzle ORM on Postgres . Tailwind + Radix primitives
  One deployable: the API is mounted inside the same server that serves the SPA.

DESIGN (non-negotiable)
  Implement Part VII exactly: minimal, modern, glass. Neutral near-monochrome
  base, ONE accent, translucent layered surfaces, generous space, one
  type family, 8px rhythm. Do NOT reproduce the old hand-drawn notebook
  styling — no wobbly borders, no paper textures, no offset "sketch" shadows.

RULES
  1. Every AI action is charged to the acting user's credit balance,
     regardless of role. No free path for admins.
  2. A failed or unusable generation is refunded or never debited.
  3. Boot migrations are additive-only and idempotent; the app must serve
     requests correctly while a migration is still in flight.
  4. All user-facing strings go through the translation function from day one.
  5. Every list endpoint filters by visibility IN SQL, never after the fact,
     so pagination limits count only rows the viewer may see.

ORDER OF WORK
  Follow Part IX. Do not start a later milestone before the previous one's
  acceptance criteria pass.

DELIVERABLE PER MILESTONE
```

```
Working code, typecheck clean, lint at or below baseline, and a short note
on what you verified in a real browser.
```

1.2 Notation

Convention	Meaning
`public`	Callable by anyone, signed in or not. `ctx.user` may be undefined.
`authed`	Requires a valid session. Throws `UNAUTHORIZED` otherwise.
`mod`	Requires role `moderator` or `admin`.
`admin`	Requires role `admin`.
🪙	A credit (called a "coin" in the UI). The platform's internal currency.
🎟	A customization ticket. A prepaid entitlement for one bounded generation.

PART I · CHAPTER 2

What the product is

> One sentence: **Modular-Start turns a subject into a repository of units and lessons, then teaches each lesson as a slide deck that quizzes as it goes — and remembers what it already taught.**

2.1 The core loop

```
┌──────────────┐
│  A SUBJECT   │  "Intermediate Spanish"  ·  "Our dinner menu"  ·  "How our CRM works"
└──────┬───────┘
       │  AI drafts the structure (lessonPath prompt)
       ▼
┌──────────────┐
│     REPO     │  title · description · template · language · banner
│   ├─ UNIT    │  a topic area / menu section / service group
│   │   ├─ LESSON      title + OBJECTIVE (the objective IS the generation prompt)
│   │   ├─ LESSON
│   │   └─ IMAGE ITEM  a picture that sits between lessons
│   └─ UNIT
└──────┬───────┘
       │  each lesson feeds the deck engine, seeded with course memory
       ▼
┌──────────────┐
│  SLIDE DECK  │  N slides · each = components + optional evaluation
└──────┬───────┘
       │  a person plays it; answers are graded (AI grades typed/solve)
       ▼
┌──────────────┐
│     RUN      │  score · elapsed · per-slide detail · annotations
└──────┬───────┘
       │  writes a LESSON LOG back to the repo
       ▼
┌──────────────┐
│ COURSE MEMORY│  "previously taught" — injected into the NEXT lesson's prompt,
└──────────────┘  with hard anti-repetition rules
```

That last arrow is the product. Anyone can generate slides; almost nothing generates lesson 7 knowing exactly what lessons 1–6 already covered and being forbidden from re-teaching it. Preserve this.

2.2 The five things a repo can be

A repo carries a `template`, which resolves to a `purpose`, which changes the deck engine's entire mode. This is the mechanism that lets one platform run a school and a restaurant.

Template	Purpose	Deck mode	Quizzes?	Ends on
`course`, `other`	education	Teaching: introduce → develop → apply	Yes	Score
`restaurant`, `shop`, `service`	commercial	Showcase: one item, persuasive, 3–4 slides	Never	Contact / order screen
`walkthrough`	walkthrough	Explain fully, same rigour as a lesson	Never	Author card
`news`	news	Newspaper briefing from a chosen moment in time	Never	Author card

2.3 Who is who

Role	Becomes one by	Can
`user`	Registering. Starts with 50 🪙.	Browse, play, generate (paying coins), build own repos, request tickets.
`moderator`	**Holding credits.** The role follows the balance automatically, in both directions.	Everything above, plus: hold and gift tickets, credit/deduct other users' coins, approve payments, flag runs, recalibrate decks.
`admin`	Granted.	Everything. Sells tickets to moderators, sets prices, verifies users, owns the marketing surfaces, publishes to the feed.

> **DESIGN DECISION WORTH KEEPING**
>
> Role is not an independent field you edit — it is a *function of the credit balance*. Every credit and debit flows through one function, and that function promotes a user to moderator the moment their balance goes positive and demotes them when it hits zero. Admins are exempt in both directions. Verification (the check mark) rides on the moderator role and is cleared automatically on demotion. This removes an entire class of "user has permissions they shouldn't" bug, because there is exactly one place the role can change.

PART II · CHAPTER 3

Stack and repository layout

3.1 Stack

Layer	Choice	Why
UI	React 19, TypeScript, Vite	—
Routing	`react-router` v7, single `<Routes>` table	Every route in one readable file.
Transport	tRPC v11 + superjson	End-to-end types with no codegen step; `Date` survives the wire.
Server cache	TanStack Query v5 via `@trpc/react-query`	Optimistic updates, invalidation, retries.
HTTP server	Hono on Node (`@hono/node-server`)	Small; runs identically on a server and on a serverless function.
ORM	Drizzle	SQL-shaped, typed, no runtime magic.
Database	Postgres, one named schema	Neon in production; a local cluster in dev.
Styling	Tailwind + CSS custom properties	Tokens live in CSS vars so theming is not a rebuild.
Primitives	Radix UI	Accessible dialogs/menus/popovers without writing focus traps.
Validation	Zod, shared between client and server	One schema validates the form and the endpoint.
Auth	JWT bearer + bcrypt	Stateless; no session table.
Tests	Vitest; Playwright for browser checks	—

3.2 Layout

```
app/
├── index.html                SPA entry — title, favicon, theme-color, meta
├── contracts/                SHARED. Imported by BOTH client and server.
│   ├── types.ts                every cross-boundary type + tuning constants
│   ├── site.ts                 the site's identity (name, slug, tagline, bio)
│   ├── slide-templates.ts      the layout catalogue + component vocabulary
│   ├── languages.ts            content languages + the language directive
│   ├── post.ts                 post categories, visibility, scopes
│   ├── grade.ts                grading thresholds
│   └── stem.ts                 STEM-topic detection
├── db/
│   └── schema.ts             Drizzle table definitions (one named pg schema)
├── api/
│   ├── boot.ts               process entry
│   ├── server.ts / app.ts    Hono app; mounts tRPC + the SPA
│   ├── context.ts            attaches ctx.user from the Bearer token
│   ├── middleware.ts         createRouter, publicQuery
│   ├── procedures.ts         authed / moderator / admin tiers
│   ├── router.ts             the root router — mounts all 21 sub-routers
│   ├── routers/*.ts          one file per domain
│   ├── ai/
│   │   ├── prompts.ts          prompt builders + Zod output schemas + repair
│   │   ├── provider.ts         key resolution, all provider adapters
│   │   └── mock.ts             offline deck generator for dev/CI
│   ├── cost.ts               estimateCost, ticketPrice
│   ├── tokens.ts             applyTokenDelta — THE ONLY credit mutation
│   ├── tickets.ts            ticket issue / spend / transfer
│   ├── finance.ts            provider pricing, usage, budgets
│   ├── settings.ts           key/value app settings
│   └── lib/migrate-required.ts  additive boot migrations
├── src/
│   ├── App.tsx               the route table
│   ├── components/           shell, design system, feature components
│   ├── pages/                one file per route
│   ├── providers/            trpc, auth, i18n, cost-confirm
│   ├── hooks/
│   └── lib/                  i18n, pdf writer, qr, canvas layout, zip
└── scripts/                  seeds, schema init, i18n coverage
```

THE CONTRACTS RULE

`contracts/` is the only directory both sides import. The client must never import from `api/` or `db/`; the server must never import from `src/`. Enforce it with a lint rule. It is what keeps the type-sharing honest instead of accidentally shipping the database driver to the browser.

PART II · CHAPTER 4

Request lifecycle, auth and roles

4.1 The lifecycle

```
browser
  │  fetch POST /api/trpc/repos.create   Authorization: Bearer <jwt>
  ▼
Hono
  │  routes /api/trpc/* to the tRPC handler; everything else to the SPA
  ▼
createContext()
  │  read Bearer → verify JWT → load the user row → ctx.user (or undefined)
  ▼
procedure tier
  │  public  → pass
  │  authed  → ctx.user or UNAUTHORIZED
  │  mod     → role in {moderator, admin} or FORBIDDEN
  │  admin   → role === admin or FORBIDDEN
  ▼
zod .input()  — rejects malformed payloads before any handler code runs
  ▼
handler  →  Drizzle  →  Postgres
  ▼
superjson serialises the result (Dates stay Dates)
```

4.2 Auth

- Password hashed with bcrypt. Never returned, never logged.
- On register or login the server issues a JWT carrying `{ id, email, role }` and the client stores it in `localStorage`.
- The *token is a hint, the database is the truth.* Context re-reads the user row on every request, so a role or balance change takes effect on the next call — not on the next login. Do not read the role out of the JWT claims.
- `auth.me` returns the session user or null; the client bootstraps from it.
- Guest mode is real: most read endpoints are `public`. A guest can browse the feed, the gallery, repos and users, and can play public decks. A guest cannot generate anything.

4.3 The one credit function

Everything that moves credits calls `applyTokenDelta(userId, delta, reason)`. It runs in a transaction and does four things atomically: check the balance can cover a debit, write the new balance, write a ledger row, and reconcile the role. Nothing else in the codebase may write `users.tokenBalance`.

```
async function applyTokenDelta(userId, delta, reason): Promise<number> {
  return db.transaction(async (tx) => {
    const user = await tx.query.users.findFirst({ where: eq(users.id, userId) });
    if (!user) throw new TRPCError({ code: "NOT_FOUND" });

    const next = user.tokenBalance + delta;
    if (next < 0) throw new InsufficientTokensError(user.tokenBalance, -delta);

    // Role follows credits, in both directions. Admins are exempt.
    const promote = user.role === "user"      && next >  0;
    const demote  = user.role === "moderator" && next <= 0;
    const role = promote ? "moderator" : demote ? "user" : null;

    await tx.update(users)
      .set({ tokenBalance: next,
             ...(role   ? { role }             : {}),
             ...(demote ? { verified: false }  : {}) })   // verification rides on the role
      .where(eq(users.id, userId));

    await tx.insert(tokenLedger).values({ userId, delta, reason, balanceAfter: next });
    return next;
  });
}
```

INVARIANT

The ledger must be able to reconstruct any balance by summing deltas. If a code path changes a balance without writing a ledger row, the Finance page silently lies. Free ticket grants are deliberately a *separate* function that does **not** write a coin-ledger row, precisely because no money moved.

PART II · CHAPTER 5

Data model

All tables live in one named Postgres schema. There are deliberately **no foreign-key constraints** — deletion order is handled explicitly in application transactions. This was a pragmatic choice for a schema that changed weekly; if you rebuild with a settled model, adding FKs is an improvement, but then you must order every cascade correctly.

5.1 Identity and money

Table	Columns	Notes
`users`	`id`, `email` uniq, `passwordHash`, `name` (≤20 chars, one word), `role`, `tokenBalance` default 50, `ticketBalance` default 0, `verified`, `avatarImageId`, `avatarVariant`, `language`, `whatsapp`, `socials` json, `contactNote`, `createdAt`	Username is one word, spaces closed up as you type. `language` is the UI language preference.
`tokenLedger`	`id`, `userId`, `delta`, `reason`, `balanceAfter`, `createdAt`	Append-only. The audit trail for every coin.
`payments`	`id`, `userId`, `packId`, `packTokens`, `amountCents`, `note`, `status` (pending/credited/rejected), `resolvedBy`, `resolvedAt`	Manual payment flow: the buyer pays off-platform and files a note; a moderator approves.
`tickets`	`id`, `repoId` (nullable), `holderId`, `issuedById`, `consumed`, `consumedAt`, `createdAt`	One row = one entitlement. `repoId` null = general (spendable on a standalone deck).
`ticketRequests`	`id`, `requesterId`, `ownerId`, `repoId`, `count`, `note`, `status`, `resolvedAt`	A user asking a repo owner for tickets.
`apiKeys`	`id`, `userId`, `provider`, `capability`, `apiKey`, `createdAt`	Bring-your-own-key. A user's own text key zeroes the text portion of their cost.
`settings`	key PK, `valueJson`	Key/value. Holds `app` (prices, packs, flags, platform keys), `finance.pricing`, `finance.usage`, `finance.budgets`.

5.2 Content

Table	Columns	Notes
`repos`	`id`, `slug` uniq, `ref` (5-char code), `title`, `description`, `template`, `ownerId`, `studyToolSlug`, `source` (ai/human), `contentLanguage`, `isPublic`, `bannerImageId`, `bannerPrompt`, `createdAt`	`ref` is a stable 5-char hash of the slug, shown as a sticker so people can refer to a repo out loud.
`units`	`id`, `repoId`, `title`, `orderIndex`	—

`lessons`	`id`, `unitId`, `title`, `objective`, `orderIndex`, `globalSeq`, `presetJson`, `presetAt`, `answerKeyGenerations`	**`objective` is the generation prompt.** `globalSeq` is the lesson's position across the whole repo — course memory is "everything with a lower globalSeq".
`unitImages`	`id`, `unitId`, `imageId`, `caption`, `orderIndex`	A picture as a unit item, interleaved with lessons.
`slideTools`	`id`, `slug` uniq, `name`, `description`, `topic`, `instructions`, `defaultSlideCount`, `defaultLevel`, `defaultImageStyle`, `defaultTone`, `template`, `contentLanguage`, `ownerId`, `source`, `isPublic`, `bannerImageId`, `repoSlug`, `repoLessonSeq`	A reusable generator. Standalone, or mirrored out of a repo lesson.
`slideTemplates`	`id`, `name`, `components` json, `tags` json, `levels` json	The layout catalogue. Admin-editable; ships with built-ins.
`slideImages`	`id`, `ownerId`, `mime`, `data` (base64), `createdAt`	Blob store in the DB. Fine at this scale; move to object storage past it.
`assignments`	`id`, `targetType`, `targetSlug`, `userId`, `assignedById`	Hand a repo / tool / post to a named person.
`favorites`	`id`, `userId`, `targetType`, `targetSlug`	Saves and follows.

5.3 Play and outcomes

Table	Columns	Notes
`runs`	`id`, `toolSlug`, `userId`, `playerName`, `deckJson`, `answersJson`, `annotationsJson`, `slideCount`, `scoreCorrect`, `scoreTotal`, `elapsedSec`, `flagged`, `isAnswerKey`, `completedAt`	The deck is stored *with* the run, so a replay is byte-identical years later even if the tool changed.
`lessonLogs`	`id`, `lessonId`, `userId`, `runId`, `slidesJson`, `scoreCorrect`, `scoreTotal`, `elapsedSec`	The record course memory is built from.
`customizations`	`id`, `userId`, `repoId`, `lessonSeq`, `toolSlug`, `deckJson`	"My version" of someone else's lesson.
`orders`	`id`, `ownerId`, `buyerId`, `buyerName`, `itemTitle`, `note`, `seen`, `createdAt`	A lead from a commercial deck's contact screen.

5.4 Marketing

Table	Columns	Notes
`posts`	`id`, `slug` uniq, `ownerId`, `caption`, `category`, `imageIds` json, `audioId`, `width`, `height`, `visibility`, `contentLanguage`, `createdAt`	An Instagram-style carousel. Slides are pre-rendered PNGs.

`castModels`	`id`, `ownerId`, `name`, `headline`, `sheet`	A recurring character used across carousels. `sheet` is the description the image prompt embeds.
`castPortraits`	`id`, `modelId`, `imageId`	—
`marketingProfiles`	`id`, `ownerId`, brand json	Saved brand: colours, logo, defaults.
`cardVersions`	`id`, `ownerId`, `name`, `kind` (business/payment), card json, `createdAt`	Named snapshots of a business card, restorable.

PART II · CHAPTER 6

Migration strategy

The system runs on serverless functions with cold starts. The boot migration is *fire-and-forget*: the server starts serving requests before it finishes. That produces one hazard and one rule.

> **THE HAZARD**
>
> On a cold start, a request can arrive while a `CREATE TABLE IF NOT EXISTS` is still in flight. The query fails with "relation does not exist", and the UI shows an empty list rather than an error — indistinguishable from "you have nothing saved". This actually happened, and cost a day of debugging a feature that was working.

The rules that fix it:

1. Migrations are **additive and idempotent only**: `ADD COLUMN IF NOT EXISTS`, `CREATE TABLE IF NOT EXISTS`, `CREATE INDEX IF NOT EXISTS`, widening a type, dropping a `NOT NULL`. Never a destructive change at boot.
2. Any router touching a **late-added** table calls a memoised `ensureTable()` before its first query, and un-memoises on failure so the next request retries. Self-healing beats correct ordering.
3. The UI must **distinguish empty from failed**. A list that got an error shows the error, in the accent-danger colour, with the server's message. "None yet" is reserved for a query that succeeded and returned nothing. This is a design requirement, not a nicety.

```
let ensured: Promise<void> | null = null;

function ensureCardVersions(): Promise<void> {
  if (!ensured) {
    ensured = db.execute(sql`CREATE TABLE IF NOT EXISTS ...`)
      .then(() => db.execute(sql`CREATE INDEX IF NOT EXISTS ...`))
      .then(() => undefined)
      .catch((e) => { ensured = null; throw e; });   // un-memoise so we retry
  }
  return ensured;
}
```

PART III · CHAPTER 7

Procedure conventions

7.1 Rules every endpoint follows

- **Validate at the edge.** Every procedure declares a Zod `.input()`. No handler inspects raw input.
- **Filter visibility in SQL.** Never fetch-then-filter: the `limit` must count only rows the viewer can see, or pagination silently drops content.
- **"Not here" beats "not allowed".** A private object returns `NOT_FOUND` to someone who may not see it, so the API does not confirm the existence of things by URL-guessing.
- **Charge late, refund eagerly.** Debit as close as possible to the irreversible work; if a later step fails, refund in the catch.
- **Machine-readable error prefixes.** Messages start with a stable token the client can branch on — `INSUFFICIENT_TOKENS:`, `NOT_ENOUGH_TICKETS:`, `AI_UNAVAILABLE:` — followed by prose a human can read. Both audiences, one string.

7.2 Shared tuning constants

Constant	Value	Meaning
`MAX_DECK_SLIDES`	15	Hard cap on a single generation.
`TICKET_DECK_LIMITS`	`{ maxSlides, maxLevel }`	What one ticket covers. The ticket price is derived from this so price and entitlement cannot disagree.
`USERNAME_MAX_LENGTH`	20	One word.
`LEVELS`	A0 A1 A2 B1 B2 C1 C2	CEFR reading bands.
`TEXT_GRADE_COST`	1 🪙	AI grading of one typed/solve answer.

PART III · CHAPTER 8

Full endpoint catalogue

21 routers. Tier notation from §1.2.

auth

Procedure	Tier	Does
`register`	public	Create account, hash password, issue JWT, seed 50 🪙.
`login`	public	Verify, issue JWT. Accepts email *or* username.
`me`	public	Session user or null.
`logout`	authed	Client-side token discard.
`updateProfile`	authed	Username, contact fields, optional password change.
`setLanguage`	authed	Persist UI language.

generate — the AI surface

Procedure	Tier	Does
`estimate`	public	Cost of a hypothetical deck, with a human-readable breakdown. Never charges.
`lessonPath`	authed	Draft a whole repo (units + lessons + a linked tool) from a description, optionally from an uploaded reference file.
`slides`	public*	**The main event.** Generate a deck. *Public in tier but throws UNAUTHORIZED — there is no anonymous AI. See §10.
`slideImage`	public	Generate one slide's image, separately, so a deck streams in.
`gradeTyped`	public	AI-grade a typed or solve answer against the reference. 1 🪙.
`gradeVisual`	public	Grade a drawn/worked answer via the vision path.
`mathSteps`	public	Expand a worked solution step by step on demand.
`tuneSettings`	public	Suggest level / slide count / style from a topic.
`recalibrateProse`	mod	Rewrite an existing deck's prose to a different density or level.
`retimeNewsSlide`	mod	Re-report one news slide from a different moment in time.

repos

Procedure	Tier	Does
`list`	public	The shelf. Filters: mine/all, category, language, favourites, following.
`getBySlug`	public	Full detail: units, lessons, images, progress, presets.

`create`	authed	From an AI draft or by hand.
`update` / `delete`	authed	Owner or admin. Delete cascades in one transaction.
`generateBanner`	authed	Charged AI banner image.
`toggleFavorite`	authed	—
`ensureStudyTool`	authed	Create the mirrored slide tool if missing.
`lessonRuns`	public	Every run against one lesson.
`courseMemory`	public	What has been taught so far — the memory injected into the next prompt.
`setLessonPreset` / `updateLessonPreset` / `deleteLessonPreset` / `lessonPreset`	authed / public	A saved generation configuration pinned to a lesson, so replays are reproducible.
`saveMyCustomization` / `myCustomization`	authed	The viewer's own version of someone else's lesson.

slideTools · units · lessons · unitImages

Procedure	Tier	Does
`slideTools.list` / `getBySlug`	public	Same filter vocabulary as repos.
`slideTools.create` / `createManual` / `update` / `delete`	authed	—
`slideTools.saveDeck` / `deck`	authed / public	Persist and read a hand-built deck.
`units.create|update|delete`	authed	—
`lessons.create|update|delete|reorder`	authed	Reorder maintains `globalSeq` — course memory depends on it.
`unitImages.create|replace|remove|setCaption|move|generateBanners`	authed	`generateBanners` is charged.

runs

Procedure	Tier	Does
`complete`	public	Write a finished run + its lesson log. Guests may finish a public deck.
`get` / `replay`	public	Read a run; replay returns the stored deck.
`listGlobal` / `listMine`	public / authed	—
`createAnswerKey` / `answerKeyFor`	authed / public	A perfect run generated from the saved preset, so anyone can study the answers.
`setFlagged`	mod	Flag a suspicious run.

posts — the feed

Procedure	Tier	Does
`list`	public	Scope `feed` (admin posts + yours + assigned) or `all` (gallery). Category, language, saved, owner filters. See §17.
`bySlug`	public	One post; NOT_FOUND if not visible.
`uploadSlide`	admin	Park one rendered PNG. Called once per slide — six 1080px PNGs in one body exceeds the request cap.
`create`	admin	Publish the carousel.
`setVisibility` / `setLanguage` / `remove`	authed	Owner or admin.
`recipients` / `setRecipients`	authed	Send a post to named people.
`toggleSaved`	authed	—

users

Procedure	Tier	Does
`directory`	public	The creator directory with search.
`profile`	public	One creator: their repos, tools, posts, badges.
`paymentCard`	public	Their public payment card, if published.
`toggleFollow` / `followingIds`	authed / public	—
`avatarQuote` / `setAvatar`	authed	Upload, or a charged AI portrait.
`list` / `detail`	mod	Admin user table.
`creditTokens`	mod	**Give or take credits**, with a reason, written to the ledger. See §15.
`createUser` / `updateIdentity` / `setRole` / `setVerified` / `deleteUser`	admin	Delete cleans every referencing table explicitly.

tokens · payments · tickets · orders

Procedure	Tier	Does
`tokens.balance`	authed	Balance + recent ledger.
`tokens.packs`	public	The purchasable packs.
`payments.submitPaidNote`	authed	"I paid" → pending row.
`payments.mine` / `pending`	authed / mod	—
`payments.approve` / `reject`	mod	Approve credits the pack and may promote the buyer.

`tickets.price` / `spendable` / `availableFor` / `mine`	authed	—
`tickets.sellToModerator`	admin	Debits coins at the live price, grows the pool.
`tickets.grantFree`	admin	Adds tickets without a coin-ledger row.
`tickets.grantToUser` / `send`	mod	Gift from the pool / transfer to another moderator.
`tickets.request` / `myRequests` / `incoming` / `resolveRequest`	authed	Ask an owner for tickets.
`orders.create` / `listMine` / `unseenCount` / `markAllSeen`	public / authed	Leads from commercial decks.

marketing · cast · keys · templates · tts · admin · finance · assignments

Procedure	Tier	Does
`marketing.storyboard`	admin	Write a carousel: per-slide title, subtitle, image brief, keywords, cast. Charged.
`marketing.mathboard`	admin	Work a problem across a carousel, one slide per phase, maths in Unicode not LaTeX.
`marketing.generate` / `highlight` / `music` / `logo`	admin	Images, keyword highlighting, an ElevenLabs music bed, a logo.
`marketing.card` / `saveCard` / `draftCard`	admin	The business card. See §22.
`marketing.cardVersions` / `saveCardVersion` / `renameCardVersion` / `deleteCardVersion`	admin	Named restorable snapshots.
`marketing.brand` / `saveBrand` / `quote`	admin	—
`cast.list` / `portrait` / `fromPhoto` / `update` / `remove`	admin	Recurring characters; `fromPhoto` uses the vision path.
`keys.list` / `upsert` / `remove` / `test`	authed	BYOK management. `test` does a live round-trip.
`templates.list`	public	The layout catalogue.
`templates.create` / `update` / `delete`	authed*	Surface is admin-gated in the route table.
`tts.status` / `voices` / `speak`	public	Read-aloud.
`admin.dashboard`	mod	Counts and totals.
`admin.aiStatus` / `getSettings` / `updateSettings`	admin	Provider diagnostics; prices, packs, flags, platform keys.
`finance.overview`	admin	Real USD spend per provider vs budget, from live model pricing.
`finance.setPricing` / `refreshPricing`	admin	Re-fetch the public model price feed.

`finance.grantTokens` / `userLedger` / `receipt`	admin	—
`assignments.assign` / `listFor` / `unassign`	authed	Hand work to a named person.

Providers and key resolution

The AI layer is multi-provider by design, because no single key is reliable and because users bring their own.

Capability	Providers
`text`	OpenAI-compatible, Anthropic, Gemini, Grok, DeepSeek, Moonshot/Kimi, OpenRouter
`image`	Gemini, OpenAI, Leonardo, Unsplash (stock fallback), mock
`tts` / music	ElevenLabs
`vision`	Gemini, OpenAI, Anthropic

9.1 Resolution order

1. **The user's own key** for that capability, if they have one. This is the only thing that changes the price: a user with their own text key pays 0 🪙 for the text portion.
2. **Platform keys** from settings.
3. **Environment keys.**
4. **Mock**, only when explicitly enabled by env var — for dev and CI.

9.2 Rotation and diversity

When several keys are available, candidates are ordered for *diversity*: one key per provider first, then the rest. A deck whose first provider is rate-limited then falls through to a genuinely different provider rather than to a second key on the same exhausted account. Failures are retried down the list; the last error is surfaced only if every candidate fails.

> **REBUILD NOTE**
>
> Keep the `AI_UNAVAILABLE:` error distinct from a content failure. "No provider answered" and "the model wrote something unusable" need different messages and different billing treatment: the first must never charge, the second may charge after repair succeeds.

9.3 Cost telemetry

Every provider call records `{ providerId, model, inputTokens, outputTokens, calls }` into `finance.usage`. The Finance page multiplies that by a live model price table (seeded from a public feed, refreshable) to show real USD spend against a per-provider budget. Budgets store a *baseline* snapshot of usage at the moment they were set, so spend counts from then rather than from the beginning of time.

PART IV · CHAPTER 10

The deck engine prompt

> This is the most valuable thing in the system. It is not one prompt — it is a prompt *assembled* from nine independent dials, and the assembly order matters.

10.1 The inputs

Input	Values	Effect on the prompt
`purpose`	education / commercial / walkthrough / news	Swaps in an entire mode block and changes whether quizzes exist at all.
`level`	A0…C2	Vocabulary and sentence complexity, *exercise difficulty*, and the paragraph floor.
`tone`	neutral, casual, conversational, friendly, professional, formal, scholarly	Register and jargon load, layered *on top of* level. Explicitly does not change how much is taught.
`textDensity`	minimal / brief / standard / explained / dense	Hard word-count floors per slide. Shifts the paragraph floor by −99/−1/0/+2/+3.
`imageStyle`	sketch / watercolor / flat / photo / none	`none` forbids image components entirely.
`subject`	auto / stem / humanities	Which layout catalogue is offered. Author's choice beats topic detection.
`previouslyTaught`	text or null	Injects the anti-repetition block.
`layoutTemplates`	filtered catalogue	The *only* slide shapes the model may emit.
`language`	ISO code	Prepends the language directive. Image prompts stay English.

10.2 Assembly order

```
[language directive]                ← first, because it governs every string
"You are the {SITE} {mode} engine…"
[MODE BLOCK]                        ← commercial | walkthrough | news, or nothing
[TEXT AMOUNT directive]             ← hard word floors, stated as rejection rules
[SPECIFICITY rule]                  ← "a paragraph that survives swapping the topic is a failed paragraph
[numbered RULES 1–9]                ← structure, palette, subject gating, evaluation kinds,
                                      image style, CEFR calibration, paragraph floor, tone
[PREVIOUSLY TAUGHT block]           ← only when course memory exists
[SLIDE LAYOUT TEMPLATES]            ← the closed catalogue + per-flavour guidance
[OUTPUT: strict JSON shape]         ← last, so it is the freshest instruction
```

10.3 The techniques that actually made output better

These are hard-won. Reproduce them.

Floors stated as rejection rules, not targets

Models systematically under-write against a soft target. "Aim for 240 words" produces 90. The fix is to state the floor as a failure condition:

```
"HARD RULE: every teaching slide carries AT LEAST 240 words of written prose —
 aim for 240-320 words across three meaty paragraphs. A slide below 240 words
 is a FAILED slide and will be rejected."
```

Word floors by density: minimal = a two-sentence *cap*; brief ≥ 90; standard ≥ 240; explained ≥ 400; dense ≥ 650.

A closed layout catalogue

The model is not free to invent slide shapes. It is handed a filtered list of named layouts — *"Text · Image · Text · Multiple choice"* — and told to build every slide as one of them, in order, dropping no step. Three consequences:

- Every layout pairs a visual with a text step, so **a slide is never just a picture and a question**. That single structural rule fixed the most common quality complaint.
- Output can be *validated* against the same catalogue — a slide that does not conform to an approved configuration is caught mechanically.
- Repeated steps mean genuinely different content: three "Image" steps means three *different* illustrations, three "Graph" steps three *different* charts. The model is told so explicitly, because its default is to restate.

The specificity rule

```
"A paragraph that would still make sense with the topic swapped out is a
 failed paragraph — rewrite it with real substance about THIS topic."
```

This one line eliminates most filler prose ("look closely at the diagram…").

Anti-repetition that says *use it*, not just *don't repeat it*

```
PREVIOUSLY TAUGHT in this course — treat every item below as slides that
already exist earlier in this same course. HARD anti-repetition rules:
(a) NEVER re-define, re-introduce, or re-explain anything listed below — not
    even "as a reminder". Reference it by name in at most ONE short clause.
(b) When a prior concept is needed, USE it — apply it, extend it, contrast it
    with the new material — exactly like a university course refers back to
    week 1 instead of re-teaching it.
(c) Every paragraph you write must teach material NOT covered below.
```

Clause (b) is the one that matters. A pure prohibition makes the model avoid the topic; telling it what to do *instead* makes it build on it.

Level controls difficulty, not just vocabulary

Stating "A1 = simple words" produced simple words wrapped around identical problems. The prompt now maps the CEFR band onto *exercise difficulty* explicitly: A0/A1 = a one-step problem with small whole numbers; A2/B1 = two-to-three steps; B2 = multi-step with abstraction; C1/C2 = genuinely challenging, multi-concept, with edge cases.

Answer-position shuffling

Models overwhelmingly place the correct option at index 0. Without a shuffle, every answer in every deck is "A". Post-process with a Fisher–Yates over the option *indices* (duplicate-text safe), then re-point `correctIndex`. Make the RNG injectable so tests are deterministic.

10.4 Evaluation kinds

Kind	Shape	Graded by
`mcq`	Exactly 4 options, one correct	Client
`mcq2`	Exactly 2 options	Client
`fillblank`	`___` in the question, `answer` + `acceptableAnswers`	Client, tolerant match
`typed`	Open short answer + a reference `answer`	**AI**, 1 🪙
`solve`	A definite problem; `answer` is the final result, `explanation` the worked method	**AI**, 1 🪙

10.5 The payment gate

Before any model call, in this exact order:

1. Signed in? No → `UNAUTHORIZED`. There is no anonymous generation.
2. Is a **ticket** valid for this job *and* is the deck within `TICKET_DECK_LIMITS`? Then reserve one — but do not consume it yet.
3. Otherwise `estimateCost()`. If the balance cannot cover it, throw `INSUFFICIENT_TOKENS:` with a message that *names the other door* — "a customization ticket would cover this deck too", or "a ticket covers up to N slides at level L — shorten the deck to use one instead". A refusal that does not say what would have worked reads as arbitrary.
4. Debit coins now; consume the ticket only on success, so a failed generation costs nothing.

Situation	Pays with
Customizing someone else's repo lesson	Ticket first, else coins
A standalone deck from a slide tool	Ticket first, else coins
The owner building their own repo	**Always coins.** A gifted ticket must not fund repo authorship, or the ticket economy leaks.

PART IV · CHAPTER 11

The other five prompts

11.1 Lesson path — drafting a whole repo

Takes a description, a template, unit and lesson counts, and optionally text extracted from an uploaded file (a menu, a syllabus, a catalogue). Returns the whole structure.

```
You are the {SITE} lesson-path architect. Draft a complete repository structure.

SUBJECT: {description}
TEMPLATE: {template} — {template guidance}
[ATTACHED REFERENCE MATERIAL — the user uploaded this; build the units, lessons
 and objectives FROM IT, staying faithful to its actual items, sections, names
 and details (do not invent items it doesn't contain): """{text, ≤12000 chars}"""]

RULES:
- Exactly {N} units, each with exactly {M} lessons.
- Lesson order must build logically (lesson N assumes lessons 1..N-1).
- Objectives are 1-3 sentence prompts that will be fed DIRECTLY to a
  slide-generation engine. Make them concrete and self-contained.
- toolName / toolTopic / toolInstructions: the linked slide tool's identity
  and 2-4 sentences of standing guidance.

OUTPUT: STRICT JSON ONLY …
```

Template guidance is what makes one architect serve five businesses:

Template	Units are…	Lessons are…	The objective is…
course	topic areas	teachable concepts	an imperative teaching prompt naming what the learner must be able to do
restaurant	menu categories	individual dishes	a story/exploration prompt: origins, ingredients, preparation, how it is served
service	service areas	individual jobs	what the service is, when it is needed, how it is done
shop	categories	products	the product's story, materials, use, care

11.2 Coach — the conversational front door

A short-reply assistant that helps someone decide what to build and returns *actions* the UI renders as buttons. Constrained hard: 2–5 sentences, plain text, no markdown, exactly one clarifying question when intent is unclear, and keyword routing ("menu/cafe/dish" → restaurant template, "repair/onboarding" → service, and so on). Output is strict JSON: `{ reply, actions[] }` where each action is `{ kind: lesson-path|slides|repos, label, payload }`.

11.3 Carousel storyboard — the marketing writer

```
You write Instagram carousels for a marketing team. The subject is {category brief}.
Given it, plan a carousel of {exactly N | as many slides as the subject genuinely
needs — at least 3, at most 10, no padding} that tells one small, complete story:
slide 1 hooks the reader, the middle slides walk through the steps or ideas one at
a time in order, and the last slide closes with a takeaway or invitation.

For EVERY slide give:
  title       — 2 to 6 punchy words, the big line on the card
  subtitle    — one or two short sentences that explain it
  imagePrompt — a concrete description of a photograph for that slide's backdrop
                (setting, subject, action, light), showing adults, no text in the picture

{cast rule} {language rule} {keyword rule}

The carousel closes with a follow card for the account posting it. Write its two
lines from the account description below, bent towards this carousel's subject.
The account: {siteBrief()}

Reply STRICT JSON ONLY: {"slides":[…],"endCard":{"headline":"…","bio":"…"}}
```

TWO RULES WORTH COPYING VERBATIM

The language rule. Everything is written in the target language *except* `imagePrompt`, which stays English because it is read by an image generator trained overwhelmingly on English captions. Translating it costs picture quality and buys nothing. Also: "any keywords you pick must be copied exactly from the text you wrote" — otherwise highlighting fails to match.

The end card is a bonus, not the deliverable. If the model skips it, the user still gets — and pays for — the carousel. Parse it with a separate `safeParse` and tolerate its absence.

11.4 Mathboard

Works a problem across a carousel, one slide per phase, with the working on the slide and the plain-language explanation in the band underneath. The model decides how many slides it takes, because the person asking does not know yet — that is why they are asking.

Maths is written in Unicode, not LaTeX. A post is a picture, exported as a PNG and read on a phone, so the notation has to be something the same canvas that draws every other slide can draw. LaTeX would mean a second renderer and a second font stack, and the preview and the export would stop agreeing — which is the one rule this tool keeps.

11.5 Grading

`typed` and `solve` answers go to the model with the question, the reference answer and the student's text, and come back as a verdict plus a short explanation. Costs 1 🪙, charged to the player. `gradeVisual` does the same through the vision path for a worked-out photo or scratchpad drawing.

PART IV · CHAPTER 12

Output repair and validation

> Real model output is imperfect in small ways. Rejecting a whole deck because one quiz had three options wastes a generation the user already paid for. The pipeline is built to salvage.

12.1 The pipeline

```
raw text
  │  extractJson()          strip fences / surrounding prose, take the outermost { … }
  ▼
repairDeckDraft()
  │  · snap an invented imageStyle to the deck's own
  │  · strip blank entries from a prose paragraphs array
  │  · DROP invalid components individually (log the type)
  │  · DROP an invalid quiz, keep the slide
  │  · a slide left with NO components is rebuilt from its own quiz,
  │    not dropped — dropping shrinks the deck below what was paid for
  ▼
zod strict parse       ← now it should pass
  ▼
ensureExplanatoryProse()
  │  any slide with no prose gets a paragraph built from its own quiz
  │  (correct answer + explanation IS real teaching content)
  │  written in the deck's own register — a menu slide gets menu copy,
  │  a news slide gets reporting, never "study what it shows"
  │  EXCEPT a Wolfram-only slide: the computation IS the explanation
  ▼
shuffleQuizAnswers()
  ▼
persist
```

12.2 Persist server-side

LEARN FROM THIS ONE

Generation used to be two calls — *generate*, then *save* — which meant a tab that navigated away mid-generation lost a deck the user had already been charged for. The endpoint now takes a `persist` flag and saves the finished deck itself, so the work is one request and completes whether or not anyone is still watching. Do this from the start.

PART V · CHAPTER 13

Coins, tickets and the ledger

> Two currencies, deliberately. **Coins** are fungible and priced per unit of work. **Tickets** are prepaid entitlements for one bounded generation, and exist so somebody can be *given* the ability to generate without being given money.

13.1 Coins 🪙

- Integer. Never negative. Balance lives on `users.tokenBalance`.
- Every movement goes through `applyTokenDelta` and writes a ledger row.
- New accounts start with **50**.
- Holding any coins makes you a moderator; hitting zero demotes you. Admins exempt.

13.2 Tickets 🎟

- One row per ticket, not a counter — so each one has an issuer, a holder, an optional repo scope, and a consumed timestamp.
- `repoId` set = scoped to that repo. `repoId` null = general, spendable on a standalone deck.
- **Spend preference.** Customizing repo X spends a ticket issued *for X* first, so the general one stays available for anything. A standalone deck spends a *general* one first, so a ticket someone was given for a specific repo is the last thing taken for an unrelated deck. Both fall through — a ticket is a ticket to the person holding it, and refusing one they own would be a technicality.
- **Consumption is atomic.** The row is claimed inside a transaction with a `consumed = false` guard, so two concurrent generations cannot double-spend the same ticket.
- Only moderators and admins *hold a pool*. An ordinary user can hold issued tickets but cannot gift them.

The pricing formula

14.1 A deck

```
base   = usingOwnTextKey ? 0 : perSlideBase × slideCount
images = imageStyle !== "none" ? perImageSlide × slideCount : 0
tts    = withTts ? perTts × slideCount : 0

total  = ceil( (base + images + tts) × levelMultiplier[level] )
```

Setting	Default	Meaning
`perSlideBase`	2 🪙	Text generation, per slide. Zeroed by a user's own text key.
`perImageSlide`	2 🪙	Per slide, when any image style is active.
`perTts`	1 🪙	Per slide, when read-aloud is requested.
`perMusic`	20 🪙	One generated music bed.
`levelMultiplier`	A0/A1 1.0 · A2 1.1 · B1 1.2 · B2 1.3 · C1 1.4 · C2 1.5	Higher levels produce more text, so they cost more.

The estimate returns a **human-readable breakdown** alongside the number — "Text: 2 🪙 × 8 slides = 16 🪙", "Images: …", "Level (C1) × 1.4". Show it. A price with no arithmetic behind it reads as arbitrary, and this is the single most common source of support questions.

14.2 A ticket

A ticket must always fully cover any in-bounds generation, so its price is the cost of the *most expensive* customization possible:

```
autoTicketPrice = max(1, ceil( (perSlideBase + perImageSlide)
                               × TICKET_DECK_LIMITS.maxSlides
                               × max(levelMultiplier) ))
```

Derived from the same cap the entitlement checks, so price and entitlement can never disagree. An admin may pin an exact override; otherwise it tracks every price change automatically.

14.3 Everything else that costs

Action	Cost
AI grading of one typed/solve answer	1 🪙
AI profile portrait	quoted by `users.avatarQuote`
Repo / tool / unit banner image	quoted at the call site
Carousel storyboard	fixed, quoted before the call
Carousel images, logo, music	per-item, quoted
Business-card AI draft	quoted by `marketing.quote`

THE STANDING RULE

"Anything that is made on the website must come out of the credits the user has — no matter if it's admin, moderator, or a normal user." There is no free path. If you add a new AI feature, it quotes first and debits through `applyTokenDelta`. Reviewers should reject any AI call that does not.

PART V · CHAPTER 15

Earn, spend, transfer

15.1 The full matrix

Movement	From → To	Endpoint	Who may	Ledger?
Signup grant	— → user	`auth.register`	automatic	Yes
Buy a pack	— → user	`payments.submitPaidNote` → `approve`	buyer files, mod approves	Yes, on approval
Manual credit	— → user	`users.creditTokens direction:"credit"`	mod, admin	Yes, with reason + actor's email
Manual deduct	user → —	`users.creditTokens direction:"deduct"`	mod, admin	Yes. Cannot go below zero.
Admin grant	— → user	`finance.grantTokens`	admin	Yes
Spend on a deck	user → —	`generate.slides`	anyone signed in	Yes
Spend on grading / images / music	user → —	various	anyone signed in	Yes
Refund	— → user	`refundTokens`	system, on failure	Yes
Buy tickets	mod's coins → mod's ticket pool	`tickets.sellToModerator`	admin sells	Yes (coin debit)
Free ticket grant	— → ticket pool	`tickets.grantFree`	admin	**No** — no money moved
Gift tickets	pool → a named holder	`tickets.grantToUser`	mod, admin	No
Transfer tickets	pool → pool	`tickets.send`	mod → mod	No
Request tickets	—	`tickets.request` → `resolveRequest`	anyone → owner	No

15.2 Guardrails

- A deduct larger than the balance is refused with the actual number: *"Sam only has 12 🪙 — can't remove 50."*
- Every manual adjustment records *who* did it, appended to the reason.
- Only moderators and admins can *receive* a ticket pool. Trying to grant one to a plain user returns a message that says what to do instead: *"Only moderators hold customization tickets — credit Sam some coins first and they become one."*
- You cannot send tickets to yourself.
- Transfer checks the sender's balance *inside* the transaction, so two simultaneous sends cannot both pass on the same last ticket.

PART VI · CHAPTER 16

Global shell and navigation

16.1 Route table

Path	Page	Access
`/`, `/feed`	Feed	public
`/feed/:slug`	Post	public if visible
`/repos`	Repos — your shelf	public
`/repos/:slug`	Repository detail	public if public
`/repos/new/manual`	Repo builder	authed
`/repos/:slug/play/:seq`	Play a lesson's preset	public
`/repos/:slug/play/:seq/edit`	Preset editor	owner
`/repos/:slug/play/:seq/mine`	Play your customization	authed
`/slides`	Slides — your tools	public
`/slides/:slug`	Slide tool — configure & generate	public
`/slides/build`, `/slides/build/:slug`	Manual deck builder	authed
`/slides/show/:slug`	Play a manual deck	public
`/gallery`	Gallery — everyone's work	public
`/users`, `/users/:id`	Directory, profile	public
`/runs`, `/runs/:runId/replay`	Runs, replay	public / owner
`/card`	Business & payment card	authed
`/credits`	**New.** Credits & tickets	authed
`/settings`	Profile, keys, preferences	authed
`/auth`	Sign in / register	public
`/about`	About	public
`/admin/*`	Dashboard, stats, controls, users, moderators, payments, flags, finance, marketing, settings	gated
`/templates`	Layout catalogue editor	admin
*	→ Feed	—

16.2 The shell

Five primary destinations. Not eleven. See §28 for why this is the single biggest intuitiveness fix.

Slot	Contents
Primary nav	**Feed · Repos · Slides · Gallery · Users**
Right cluster	Credit pill (balance, links to `/credits`) · language switch · avatar menu
Avatar menu	Profile · Card · Credits · Settings · Admin (only if admin) · Sign out
Mobile	Bottom tab bar with the same five; the right cluster collapses into the avatar sheet.

16.3 Cross-cutting behaviours

Behaviour	Rule
Cost confirmation	Any charged action opens a sheet showing the itemised estimate and the balance after. A "don't ask again" preference is remembered per browser. The *first* time in a session it always shows.
Language filter	A two-state switch in the header: **EN** and **ES**. There is no "All" — English *is* the catch-all. Selecting ES sends `language: "es"` and shows only Spanish content. Selecting EN sends no filter and shows everything. A French lesson gets a FR sticker and appears in the English shelf.
Content-language sticker	Every feed post, repo, slide tool and card carries a two-letter language chip.
Assignment beats filtering	Something handed to you by name is never hidden by a language filter. Someone made a decision about you; a filter must not quietly overrule it.
Admin annotation overlay	A floating pencil on every page lets an admin draw on it. Marks are keyed to the pathname, survive closing the toolbar (going `pointer-events: none`), and are erased on reload or navigation. Excluded from player routes, which have their own tools.
Empty vs failed	Never render "None yet" for a failed query. Failures show the server's message in the danger colour with a retry.

PART VI · CHAPTER 17

Feed

Routes `/` and `/feed` · **Access** public · **Data** `posts.list`

17.1 Purpose

The front door. An Instagram-style vertical stream of carousel posts. Deliberately **narrow**: it carries the site's own posts, your own, and anything sent to you by name. Everyone else's public work lives in the Gallery. A feed that filled with every account's advertising would stop being worth opening.

17.2 Who sees what

```
scope = "feed"  (this page)
  visible = (visibility = 'public' AND author.role = 'admin')
         OR (ownerId = viewer)
         OR (slug IN viewer's assignments)

scope = "all"   (the Gallery's posts tab)
  visible = (visibility = 'public')
         OR (ownerId = viewer)
         OR (slug IN viewer's assignments)
```

IMPLEMENTATION REQUIREMENT

This must be one SQL `WHERE`, not a post-filter. The `limit` has to count rows the viewer can actually see, or a page of 30 arrives with 4 posts on it.

17.3 What a post carries

```
PostSummary {
  slug, caption, category
  imageUrls: string[]          every slide, in carousel order
  audioUrl:  string | null     the music bed, if one was made
  width, height                the aspect the carousel was rendered at
  ownerId, ownerName, ownerAvatarUrl, ownerVerified
  createdAt
  mine, saved                  viewer-relative
  who: "public" | "private"
  contentLanguage              drives the sticker and the filter
  assignedCount                how many people it was sent to
}
```

17.4 Layout and interactions

Element	Behaviour
View toggle	Stream (one post, full width, ~600px) ↔ Grid (3-up thumbnails). Remembered.
Category chips	Course · Restaurant · Service · Shop · Walkthrough · News. Single-select, clearable.
Saved toggle	Only posts you saved. Returns empty for signed-out viewers rather than everything.
Carousel	Swipe / arrows / dots. Keyboard ← →. Counter "3 / 6".
Audio	Muted by default. One control; only one post plays at a time.
Author row	Avatar, username, verified check, language chip. Tap → profile.
Save	Optimistic. Reverts with a toast on failure.
Owner controls	Visibility, language, recipients, delete — inline, never in a separate admin screen.
Empty state	Explains the feed is narrow *and* links to the Gallery. Do not leave a first-time visitor on a blank page.

17.5 New design

- Post surface is a glass card floating on a soft neutral canvas: `--surface-glass`, 1px hairline, 20px radius, and a large soft shadow that lifts on hover.
- The image is the hero, edge-to-edge inside the card radius. Caption and controls sit *below* the image, never over it.
- Category chips are quiet pills — filled only when active. No colour-coding by category; one accent for "on".
- Skeletons match the final aspect ratio exactly so nothing reflows on load.

PART VI · CHAPTER 18

Repos

Routes `/repos`, `/repos/:slug` · **Data** `repos.list`, `repos.getBySlug`, `repos.toggleFavorite`, `repos.delete`

18.1 Purpose

A repo is a *notebook*: a structured body of material — units containing lessons and images. `/repos` is **your own shelf**; everyone else's are in the Gallery. This split is load-bearing and should survive the redesign.

18.2 The shelf

Element	Behaviour
Filters	Category (the 7 templates) · language · favourites · following · search.
Card	Banner image (or a generated gradient), title, description, owner, unit/lesson counts, progress ring, `ref` sticker, language chip, source badge (AI / hand-built), privacy.
Actions	Open · favourite · delete (owner/admin, confirmed).
Create	Two doors, equally weighted: **Describe it** (AI drafts the whole structure — quoted first) and **Build it myself** (empty repo).

18.3 Repository detail

Region	Contents
Header	Banner, title, description, owner, ref, language, counts, favourite, edit/delete for the owner.
Units	Collapsible. Each holds an ordered mix of **lessons** and **image items**, reorderable by the owner.
Lesson row	Sequence number, title, objective (the generation prompt — shown, because it is the thing that produces the deck), best score, preset indicator, answer-key link.
Lesson actions	Play preset · Customize (charged) · Play my version · Edit preset (owner) · Create answer key · View runs.
Course memory panel	What has been taught so far — the exact text injected into the next lesson's prompt. Surfacing this is what makes the product's core mechanic legible.
Tickets	How many you can spend here; request more from the owner; owner grants from their pool.

18.4 New design

- Shelf is a responsive grid of glass cards with a 16:9 banner. Hover lifts 2px and strengthens the border — no scale transform.
- Progress as a thin determinate bar along the card's bottom edge, not a ring in a corner.
- Units are a left rail on desktop (sticky, showing progress per unit) and an accordion on mobile. The rail replaces scroll-hunting through a long page.

- A lesson row's *primary* action is one button. Everything else lives behind a quiet overflow menu. Today every row shows six controls and the page reads as a control panel rather than a table of contents.

PART VI · CHAPTER 19

Slides

Routes `/slides`, `/slides/:slug`, `/slides/build`, `/slides/show/:slug` · **Data** `slideTools.*`, `generate.*`, `runs.complete`

19.1 Purpose

A *slide tool* is a saved generator: a topic, standing instructions, and defaults. Point it at a subject and it produces a deck. Tools are either standalone or mirrored out of a repo lesson.

19.2 The tool page — the configurator

Control	Options	Notes
Topic	free text	Falls back to the tool's topic, then its name.
Instructions	free text	Standing guidance.
Slide count	1–15	Drives the price linearly.
Level	A0…C2	Vocabulary, exercise difficulty, paragraph floor, price multiplier.
Image style	sketch / watercolor / flat / photo / none	`none` removes the image line from the price.
Category	the 7 templates	Sets the deck's *purpose* — this is the biggest single switch. Say so in the UI.
Advanced — collapsed by default		
Tone	7 registers	Voice and jargon load, not depth.
Text amount	minimal → dense	Show the actual word target per slide. It is the difference between a 2-sentence card and a 650-word page.
Subject	auto / STEM / humanities	Which layout catalogue is offered.
Language	15 languages	What the deck is written in.
Web search	on / off	Ground the deck in current facts first.
Include questions	on / off	Off strips every evaluation before storing.
News period	free text	News decks only — the moment the briefing reports from.
Template plan	per-slide layout pin	Index *i* → slide *i*+1; null = let the AI choose.

A **live price** sits beside the controls and updates on every change, with the itemised breakdown one tap away. Generating opens the cost sheet unless suppressed.

19.3 The player

Aspect	Behaviour
Components	prose, latex, chart (bar/line/pie/area), svg diagram, table, sticky note, image, code, Wolfram|Alpha card.
Evaluation	Per §10.4. Typed and solve go to the AI for 1 🪙 with an explicit confirm.
Scratchpad	Solve questions get a drawing surface; the work can be graded by vision.
Read-aloud	Per-slide TTS with voice selection, when a key is configured.
Annotation	Draw over slides during presentation; strokes persist into the run record.
Finish	*Education* → score + review. *Commercial* → contact/order screen writing an order lead. *Walkthrough/news* → author card.
Persistence	`runs.complete` stores deck + answers + annotations + timing, and writes the lesson log that becomes course memory.

19.4 New design

- Configurator as a two-column layout: controls left, a **live deck outline** right (N numbered cards showing the pinned or predicted layout). Today the settings are a long scroll with no picture of the outcome.
- Price is a persistent element in the sticky action bar, not a line buried in the form.
- Player chrome disappears: full-bleed slide, controls fading in on pointer move, a thin progress rail at the top. Glass only for the floating control cluster.
- Every advanced control carries a one-line plain-language consequence, always visible — not a tooltip. "Dense — about 650 words a slide, a serious reading assignment."

PART VI · CHAPTER 20

Gallery

Route `/gallery` · **Access** public · **Data** the same list endpoints with `mine: false` / `scope: "all"`

20.1 Purpose

Everything *everyone* has made. The counterweight to three deliberately narrow personal surfaces: `/feed` shows the site's posts, `/repos` and `/slides` show only your own. The Gallery is where the platform's body of work actually lives.

20.2 Structure

Three tabs — **Slides · Repos · Feed** — rendering the same components as the personal shelves with the ownership filter inverted. Reuse, do not duplicate: the Slides tab is literally the Slides shelf with `mine={false}`.

Element	Behaviour
Filters	Per tab, the same vocabulary as the personal shelf, plus **following only**.
Favourites	Marked here are yours and show on your shelves' favourite filter — but your shelves still *list* only what you created.
Language	Obeys the global switch.

20.3 New design

- A segmented control, not three chip-buttons. It is one thing being switched, and a segmented control says so.
- A one-line explainer under the tabs, permanently: *"Everyone's work. Your own shelves stay just yours."* The relationship between Gallery and the personal shelves is the single most confusing thing in the current information architecture.
- Masonry for the posts tab (variable aspect ratios), uniform grid for slides and repos.

PART VI · CHAPTER 21

Users

Routes `/users` , `/users/:id` · **Data** `users.directory` , `users.profile` , `users.toggleFollow` , `users.paymentCard`

21.1 Directory

Element	Behaviour
Search	Debounced 250ms, server-side by name and email.
Filters	Following only · creators only (repoCount > 0) · category (from the templates they publish).
Row	Avatar, username, role chip, verified check, repo count, the categories they work in.
Follow	Optimistic with rollback. Guests get "Sign in to favourite creators".

21.2 Profile

Region	Contents	Visible to
Header	Avatar, username, role, verified badge, follow button, contact (WhatsApp / socials / note) when published.	everyone
Work	Their repos, slide tools and posts — public ones only for a stranger.	everyone
Payment card	Their published payment card, if they made one.	everyone
Ticket actions	Send tickets · grant free tickets.	mod / admin
Credit actions	Credit or deduct coins with a reason.	mod / admin
Identity actions	Rename, change email, set role, verify.	admin
Buy credits	Pack list and the "I've paid" note.	self

21.3 New design

- Directory as a dense list, not cards. People are scanned, not browsed.
- **Separate the moderation rail.** Today credit, ticket and identity actions are mixed into the public profile. Move every privileged action into one right-hand "Manage" panel, clearly marked, collapsed by default. A moderator viewing a profile should see the profile first.
- Verified check is a small filled glyph next to the name with a tooltip that says what it means — "vouched for by an admin" — because nobody guesses correctly.

PART VI · CHAPTER 22

Card

Route `/card` · **Data** `marketing.card`, `saveCard`, `draftCard`, `cardVersions`, `saveCardVersion`, `renameCardVersion`, `deleteCardVersion`, `quote`

22.1 Purpose

A print-quality business card and a separate payment card, designed in the browser and exported as PNG, PDF, or a print sheet with front and back side by side.

> THE RULE THAT MAKES THIS WORK
>
> **One layout function, two renderers.** A single pure function takes the card model and a canvas measuring context and returns absolute positions for every text run, rule and image. The on-screen preview walks that output as positioned `div`s; the PNG and PDF exporters walk the same output with `fillText`. Preview and export cannot drift, because there is only one layout. Every attempt to lay out the preview in CSS and the export in canvas produced two different cards.

22.2 The model

```
CARD_W = 1050, CARD_H = 600      // print pixels; the preview scales by container query

CardPair { business: BusinessCard, payment: BusinessCard }   // fully independent

BusinessCard {
  company, name, title, tagline, details
  bg, accent, ink, inkSoft         // the palette — SEPARATE per card
  backLayout: "quote" | "contact" | "payments" | "badge"
  frontMark:  "logo" | "qr"
  logoUrl
  methods: PaymentMethod[]         // payment card only
}
```

> LEARNED THE HARD WAY
>
> The business card and the payment card must be **separate objects**. They shared one model at first, so changing the payment card's colour changed the business card's. Split them at the type level, not by convention.

22.3 Payment methods

Nine kinds, each with its own fields and a brand colour used for a vertical rule beside its block, so a card with four methods reads as four groups rather than eight lines.

Kind	Brand	Kind	Brand	Kind	Brand
Binance	`#F0B90B`	Bitcoin	`#F7931A`	Ethereum	`#627EEA`
USDT TRC-20	`#26A17B`	PayPal	`#0070BA`	Zinli	`#0ABF9E`
Pago Móvil	`#2E9E4F`	Bank (VE)	`#1E5AA8`	Cash App	`#00D632`

Crypto addresses are unbreakable strings, so the wrap is **character-level**, not word-level. A truncated wallet address is worse than useless — it must print in full.

22.4 Features

Feature	Behaviour
Two cards	Business and Payment tabs; entirely independent content and palettes.
Back layouts	Quote · Contact · Payments · Badge.
Front mark	Logo (uploaded or AI) or a QR code.
Sides	Front / back toggle; the preview shows one at a time, the exports offer both.
Exports	PNG · PDF · print sheet (front and back side by side, several cards to a page). Filenames derive from the site slug: `modular-start-business-card.pdf`.
AI draft	Charged. Writes the role line and tagline from a short description.
Versions	Named snapshots, listed below the editor, restorable and renameable. The panel must distinguish "no versions yet" from "the query failed".

22.5 New design

- Three columns on desktop: controls · a large centred preview on a neutral backdrop · the versions rail. The card is the subject of the page, so it should dominate it.
- Palette editing as swatch pickers with live contrast checking — warn when the ink fails against the background, because a card that fails contrast is unreadable in print too.
- Export as one primary button with a format segment beside it, not three separate buttons plus a fourth.

PART VI · CHAPTER 23

Credits

Route `/credits` · **Access** authed · **New page.** Money moves out of Settings entirely.

> Today, buying credits, reading the ledger, holding tickets and requesting more are scattered across Settings, the user profile, and two admin screens. This page brings the whole economy into one place: **what you have, what things cost, and how to move them.**

23.1 Sections

1 · Balance

- Large numeric balance in 🪙, and the ticket count beside it.
- A 30-day sparkline of the balance, drawn from the ledger.
- Role state, stated plainly: *"You hold credits, so you are a moderator. At zero credits you go back to being a user."* This relationship is currently invisible and surprises people.

2 · Calculator

The estimator, exposed directly rather than only inside a generation flow. Sliders and selects for slide count, level, image style, read-aloud; live total; the itemised breakdown always visible; and the effect of your own API key shown as a struck-through line rather than a silent zero.

```
Text        2 🪙 × 8 slides                  16 🪙
Images      2 🪙 × 8 slides                  16 🪙
Read-aloud  1 🪙 × 8 slides                   8 🪙
Level (C1)  × 1.4
                                        ──────────
                                   Total  56 🪙
                             Balance after  194 🪙   (you have 250 🪙)
```

Below it, a static table of every other charged action (§14.3), so nothing is a surprise.

3 · Get credits

- The purchasable packs with price and unit rate.
- The manual flow, stated honestly: pay off-platform, file an "I've paid" note, a moderator approves and the credits land. Show the pending state and the elapsed time.
- Your submitted payments with status: pending · credited · rejected.

4 · Tickets

- What a ticket is, in one line: *"A prepaid pass for one generation of up to {maxSlides} slides at level {maxLevel}. Spent instead of coins when it fits."*
- Your tickets grouped by scope — general, and per repo — with counts.
- **Request** tickets from a repo owner, with a note. Your outgoing requests and their status.

- **Incoming** requests to you as an owner: approve or decline inline.
- Moderators additionally see their *pool*, the live ticket price, buy-from-admin, gift-to-a-user and send-to-a-moderator.

5 · Transfer

One panel, permission-aware, with a single shape: *pick a person → pick what → pick how many → say why → confirm.*

You are	You may move
user	Nothing. The panel explains what would unlock it.
moderator	Credits to a user (credit or deduct, with a reason) · tickets from your pool to anyone · tickets to another moderator.
admin	All of the above · sell tickets to a moderator at the live price · grant free tickets.

Every transfer shows a confirmation naming the recipient, the amount and the resulting balances *before* committing. Every one writes a reason. Failures surface the server's exact message — those messages are written to be read by people.

6 · Ledger

- Every movement: date, reason, delta (signed, coloured), balance after.
- Filter by direction and date range; export CSV as `modular-start-ledger.csv`.
- The reason string is the whole value here — *"slides: intro-to-fungi (8 slides)"*, *"manual credit (by admin@…)"*, *"bought 3 customization tickets @ 60 🪙"*. Write reasons for the person reading the row six weeks later.

23.2 New design

- Balance as a single large glass panel at the top, full width, with the sparkline behind the number at low opacity.
- The other five sections as cards in a two-column masonry; Transfer and Tickets collapse away entirely for a plain user rather than showing a disabled wall.
- Positive deltas in the success colour, negative in the ink colour — *not* in the danger colour. Spending is normal; only failures are red.
- Tabular figures everywhere numbers are stacked.

Design principles

> The old skin was a hand-drawn notebook — wobbly borders, paper texture, offset shadows, a warm cream palette. It was distinctive and it is going away. The replacement is quiet, modern and glass: the interface recedes and the content is the only thing with colour.

Principle	In practice
One accent, everything else neutral	A single indigo carries every interactive state. Colour beyond that is content — a post's photo, a card's palette, a chart's series. If a control is coloured, it is because it is *on*.
Depth by light, not by line	Layers separate through translucency, a hairline border and a soft shadow. No heavy 2px outlines, no offset drop shadows.
Glass with restraint	Translucency for things that *float over* content: the header, sheets, popovers, player controls, the cost dialog. Content surfaces are solid. Glass everywhere is glass nowhere.
Space is the layout	Grouping comes from whitespace on an 8px rhythm before it comes from a border or a background.
One type family	A single modern grotesque across the whole product, differentiated by weight and size. The old display/body split is gone.
Say the consequence	Every control that costs money, changes visibility, or changes what the AI produces states its effect in plain language, permanently — not in a tooltip.
Motion confirms, never decorates	120–200ms, ease-out, opacity and 2–8px translation only.

Tokens

Everything is a CSS custom property on `:root`, overridden under `[data-theme="dark"]`. Tailwind reads them, so a theme change is never a rebuild.

25.1 Colour — light

canvas `#FAFAFB`	**surface** `#FFFFFF`	**surface-sunken** `#F3F4F6`	**text** `#0B0C0F`	**text-muted** `#52586B`
text-faint `#8B90A0`	**border** `#E5E7EB`	**accent** `#4F46E5`	**accent-wash** `#EEF0FF`	**success** `#059669`
warning `#D97706`	**danger** `#DC2626`			

```
:root {
  /* surfaces */
  --canvas:          #FAFAFB;
  --surface:         #FFFFFF;
  --surface-sunken:  #F3F4F6;
  --surface-raised:  #FFFFFF;

  /* glass — for things that float OVER content */
  --glass-bg:        rgba(255,255,255,0.72);
  --glass-border:    rgba(255,255,255,0.85);
  --glass-blur:      20px;
  --glass-sat:       180%;

  /* text */
  --text:            #0B0C0F;
  --text-muted:      #52586B;
  --text-faint:      #8B90A0;
  --text-on-accent:  #FFFFFF;

  /* lines */
  --border:          #E5E7EB;
  --border-strong:   #D1D5DB;
  --ring:            rgba(79,70,229,0.38);

  /* accent + semantics */
  --accent:          #4F46E5;
  --accent-hover:    #4338CA;
  --accent-wash:     #EEF0FF;
  --success:         #059669;  --success-wash: #ECFDF5;
  --warning:         #D97706;  --warning-wash: #FFFBEB;
  --danger:          #DC2626;  --danger-wash:  #FEF2F2;

  /* elevation */
  --shadow-sm: 0 1px 2px rgba(11,12,15,.05);
  --shadow-md: 0 4px 12px rgba(11,12,15,.07), 0 1px 3px rgba(11,12,15,.05);
  --shadow-lg: 0 12px 32px rgba(11,12,15,.10), 0 2px 8px rgba(11,12,15,.05);
  --shadow-glass: 0 8px 32px rgba(11,12,15,.10), inset 0 1px 0 rgba(255,255,255,.55);

  /* radius */
  --r-xs: 6px; --r-sm: 10px; --r-md: 14px; --r-lg: 20px; --r-xl: 28px; --r-full: 999px;

  /* space — 8px rhythm, 4px only for icon gaps */
  --s-1: 4px;  --s-2: 8px;  --s-3: 12px; --s-4: 16px;
  --s-5: 24px; --s-6: 32px; --s-7: 48px; --s-8: 64px;

  /* type */
  --font: "Inter var", Inter, ui-sans-serif, system-ui, -apple-system, sans-serif;
  --font-mono: "JetBrains Mono", ui-monospace, SFMono-Regular, monospace;
}
```

25.2 Colour — dark

```
[data-theme="dark"] {
  --canvas:         #0A0B0E;
  --surface:        #131519;
  --surface-sunken: #0F1114;
  --surface-raised: #1A1D23;

  --glass-bg:       rgba(22,25,31,0.68);
  --glass-border:   rgba(255,255,255,0.09);

  --text:           #F2F3F5;
  --text-muted:     #A0A6B4;
  --text-faint:     #6B7180;

  --border:         #23262E;
  --border-strong:  #313640;

  --accent:         #7C74F0;    /* lifted for contrast on dark */
  --accent-hover:   #928BF5;
  --accent-wash:    rgba(124,116,240,0.14);

  --success: #34D399; --success-wash: rgba(52,211,153,.13);
  --warning: #FBBF24; --warning-wash: rgba(251,191,36,.13);
  --danger:  #F87171; --danger-wash:  rgba(248,113,113,.13);

  --shadow-lg: 0 12px 32px rgba(0,0,0,.55);
  --shadow-glass: 0 8px 32px rgba(0,0,0,.45), inset 0 1px 0 rgba(255,255,255,.06);
}
```

25.3 Type scale

Role	Size / line / weight / tracking	Used for
display	44 / 1.05 / 700 / −0.03em	Marketing headers only
h1	30 / 1.15 / 650 / −0.02em	Page titles
h2	22 / 1.25 / 650 / −0.015em	Section heads
h3	17 / 1.35 / 600 / −0.01em	Card titles
body	15 / 1.6 / 400	Default
body-sm	13.5 / 1.55 / 400	Secondary
caption	12 / 1.45 / 500	Metadata, chips
micro	11 / 1.4 / 600 / 0.06em uppercase	Labels, eyebrows
mono	13 / 1.5	Codes, refs, wallet addresses, figures

Numbers that stack — balances, prices, ledger rows, scores — use `font-variant-numeric: tabular-nums`.

25.4 The glass recipe

```
.glass {
  background: var(--glass-bg);
  backdrop-filter: blur(var(--glass-blur)) saturate(var(--glass-sat));
  -webkit-backdrop-filter: blur(var(--glass-blur)) saturate(var(--glass-sat));
  border: 1px solid var(--glass-border);
  box-shadow: var(--shadow-glass);
  border-radius: var(--r-lg);
}

/* Fallback: without backdrop-filter, glass becomes an opaque surface.
   Never leave translucency without a blur — the text becomes unreadable. */
@supports not (backdrop-filter: blur(1px)) {
  .glass { background: var(--surface); }
}
```

Use glass for	Do not use glass for
The header (sticky over scrolling content)	Content cards on the canvas — they get `--surface`
Sheets, dialogs, popovers, menus	Form fields
Player control clusters over a slide	Tables
The cost-confirmation dialog	Anything with a long paragraph of text on it
Toasts, the mobile tab bar	Anything printed or exported

Components

26.1 The set

Component	Spec
Button	Height 36 (sm 30, lg 44). Radius `--r-sm`. *Primary*: accent fill, white text, `--shadow-sm`. *Secondary*: surface, 1px border. *Ghost*: transparent, wash on hover. *Danger*: danger fill, only for destructive confirmations. Focus is always a 2px `--ring` offset 2px.
Input / Select / Textarea	Surface-sunken fill, 1px border, radius `--r-sm`, 36px min height. Focus: border → accent + ring. Error: border → danger, message below in danger, 12px. Label above, 12px `--text-muted`. Helper text below, persistent.
Card	`--surface`, 1px `--border`, radius `--r-md`, `--shadow-sm`. Interactive: hover → `--shadow-md` + `translateY(-2px)` + `--border-strong`. Never scale.
Chip / Pill	Height 26, radius `--r-full`, 12px caption. Off: surface-sunken + faint text. On: accent-wash + accent text + accent border. Language chips are mono uppercase.
Segmented control	Sunken track, radius `--r-full`, an accent-filled thumb that slides 180ms. For 2–4 mutually exclusive views (Gallery tabs, card front/back, export format).
Sheet / Dialog	Glass. Radius `--r-xl`. Enters 180ms: opacity 0→1, translateY 12px→0. Scrim `rgba(11,12,15,.45)` with a 2px blur. Mobile: bottom sheet with a drag handle.
Cost sheet	A dedicated dialog: the action named, the itemised breakdown, the total, the balance after, a "don't ask again" checkbox, and Cancel / Confirm. The confirm button carries the number: "Generate — 56 🪙".
Empty state	Icon, one-line title, one-line explanation, one primary action. Never a bare "Nothing here".
Error state	Danger wash panel, the server's message verbatim, a retry button. **Structurally distinct from empty.**
Skeleton	Sunken blocks at the final dimensions, 1.4s shimmer. Must not reflow when real content lands.
Toast	Glass, bottom-right (bottom-centre on mobile), 4s, dismissible. Errors persist until dismissed.
Stat tile	Micro label, large tabular number, optional delta with an arrow. Used on Credits and admin dashboards.
Price display	A dedicated component: coin glyph + tabular number + an info affordance opening the breakdown. Use it everywhere a price appears so prices always look the same.

26.2 Density and grids

Context	Rule
Page gutter	16px mobile · 32px tablet · 48px desktop
Content max width	1200px; 680px for prose; 600px for the feed stream
Card grid	`repeat(auto-fill, minmax(260px, 1fr))`, gap 24px
Section spacing	48px between major sections, 24px within
Form rows	16px between fields, 24px between groups

PART VII · CHAPTER 27

Motion and accessibility

27.1 Motion

Interaction	Duration / easing	Properties
Hover	120ms ease-out	background, border, shadow, translateY(−2px)
Press	80ms	translateY(0), shadow-sm
Dialog / sheet in	180ms cubic-bezier(.16,1,.3,1)	opacity, translateY 12px
Dialog out	120ms ease-in	opacity
Segment thumb	180ms spring-ish	transform
Page transition	150ms	opacity only. Never slide whole pages.
Skeleton	1400ms linear infinite	background-position

```
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: .01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: .01ms !important;
  }
}
```

27.2 Accessibility

- **Contrast.** Body text ≥ 7:1 against its surface; secondary ≥ 4.5:1; interactive borders ≥ 3:1. Verify glass surfaces against the *worst-case* background they can float over — a white caption over a blurred photograph is the failure case, and it is why captions sit below images, not on them.
- **Focus is always visible.** 2px `--ring`, 2px offset, never removed. Test the whole app with the keyboard only.
- **Targets** ≥ 44×44px on touch, including carousel dots and chips.
- **Semantics.** Radix primitives for anything with a focus trap or roving tabindex. Tabs get `role="tablist"`; the language switch is a real radio group.
- **Announce async results.** Toasts and generation progress live in an `aria-live="polite"` region.
- **Never colour alone.** A ledger delta shows its sign; a verified badge has a glyph and a tooltip; an error has an icon and text.
- **Theme.** Respect `prefers-color-scheme`, and honour an explicit `data-theme` override in both directions.
- **Translate from day one.** Every user-facing string goes through the translation function on first write. Retrofitting ~1,600 strings afterwards is weeks of work and a permanent source of misses — this is measured, not hypothetical.

NAME THE TRANSLATOR CAREFULLY

Do not call it `t`. Codebases are full of locals named `t` — a template, a totals object, a timer handle — and an import named `t` is silently shadowed inside those scopes. A shadowed translator does not fail loudly; it calls a totals object as a function in whichever branch nobody has clicked yet. Use `say()`.

Key the dictionary on the **English source string**, not an invented id. `say("Save")`, not `say("card.actions.save")`. A missing translation then falls back to perfectly good English instead of putting `card.actions.save` on a button, and the code stays readable.

Diagnosis: what is confusing today

> These are real, observed problems with the current product — not style opinions. Each one has a fix in Chapter 29.

#	Problem	Why it happens
1	**Too many top-level destinations.** Feed, Repos, Slides, Gallery, Users, Card, Runs, About, Settings, Admin — ten or more, flat.	Every feature added a nav entry. Nothing was ever demoted.
2	**"Repo" and "Slide tool" are unexplained jargon.** A new user cannot tell which one they want, and the two shelves look identical.	The names are the developer's model, not the user's.
3	**Gallery vs the personal shelves is invisible.** People cannot find work they know exists, because `/repos` silently shows only their own.	The filter is implicit. Nothing on the page says "yours only".
4	**Cost is discovered late.** The price appears at confirmation, after the settings are already chosen.	The estimator is wired into the dialog, not the form.
5	**Advanced generation settings have invisible consequences.** "Dense" and "Standard" look like a preference; they are the difference between 240 and 650 words a slide.	Labels without stated effects.
6	**The role-follows-credits rule is a surprise.** Users are promoted and demoted silently.	Never stated in the UI.
7	**Empty and failed look the same.** A failed query renders "None yet", so a broken feature looks like an unused one.	Query error states were never given a distinct treatment. This cost real debugging time.
8	**Moderation controls sit inside public profiles.** Credit, deduct, verify and role controls are interleaved with someone's public work.	Features were added where the data already was.
9	**Course memory — the best thing in the product — is nearly invisible.**	Buried in a panel. Most users never learn the system remembers.
10	**The money is scattered.** Balance in the header, packs in Settings, tickets on profiles, ledger in admin, prices in a third place.	No page ever owned the economy.
11	**Rows carry six equal-weight buttons.** A lesson row offers play, customize, my version, edit preset, answer key, runs — all the same size.	No action hierarchy.
12	**Sign-in forms fight the browser.** A current-password field marked `autocomplete="current-password"` on a *profile* page gets autofilled, and the form read that as intent to change the password — blocking Save over a field nobody touched.	Reusing sign-in semantics on a settings form.

PART VIII · CHAPTER 29

The fixes

29.1 Structural

Fixes	Change
1	**Five primary destinations: Feed · Repos · Slides · Gallery · Users.** Card, Credits, Settings and Admin move into the avatar menu. Runs becomes a tab on the profile. About moves to the footer.
2	**Rename in the UI, keep the slugs.** "Repo" → **Course** (with the subtitle "units and lessons"). "Slide tool" → **Deck maker**. The URLs, database columns and API stay `repos` / `slideTools` — renaming those buys nothing and breaks every link.
3	**Label the ownership filter.** Every personal shelf carries a visible segmented control — **Mine · Everyone** — where "Everyone" navigates to the matching Gallery tab. The distinction stops being a secret.
10	**Ship `/credits`** (Chapter 23) and make the header balance pill link to it. Remove money from Settings entirely; Settings becomes profile, keys, and preferences.
8	**One "Manage" rail** on the profile, collapsed by default, holding every privileged action. The public profile reads as a profile.

29.2 Comprehension

Fixes	Change
4	**Live price in the form.** A persistent price element in the sticky action bar, updating on every change, with the itemised breakdown one tap away. The confirm button carries the number: "Generate — 56 🪙". No number should ever be a surprise.
5	**State the consequence under every advanced control**, permanently, not in a tooltip. "Dense — about 650 words a slide, a serious reading assignment." "Scholarly — technically precise, assumes an engaged reader." "Commercial — a showcase with no questions."
6	**State the role rule** on Credits and in the avatar menu: "You hold credits, so you are a moderator." And toast the transition when it happens, in both directions.
9	**Promote course memory.** On a repo, a permanent panel: "This course has taught 14 concepts across 6 lessons. Lesson 7 will build on them, not repeat them." Open it to read the exact text the AI will receive. On the configurator, a badge when memory is active. This is the differentiator; it should be the thing people notice.

29.3 Craft

Fixes	Change
7	**Three distinct states for every async surface:** loading (skeleton at final dimensions), empty (icon + explanation + one action), failed (danger panel + the server's message + retry). Make this a lint-able convention — a list component that cannot render a failure is incomplete.
11	**One primary action per row.** Everything else behind a quiet overflow menu. On a lesson row the primary is Play; Customize is secondary because it costs money; the rest are in the menu.
12	**Never use sign-in autocomplete semantics outside sign-in.** A profile-page password field is `autocomplete="new-password"` with a non-standard `name`. And intent to change a password is a *new* password — never a filled-in current one.

—	**Destructive actions name the thing.** "Delete *Intro to Fungi* and its 18 lessons?" — not "Are you sure?".
—	**Errors say what would have worked.** Already true of the credit messages; extend it everywhere. "A ticket covers up to 12 slides at B2 — shorten the deck to use one instead" is the model.

Build order

#	Milestone	Contains	Done when
1	**Foundation**	Vite + React + TS; Hono + tRPC + superjson; Drizzle + Postgres; the design tokens and the base component set; `say()` wired from the first string.	A themed page renders, light and dark, and `ping` round-trips through tRPC.
2	**Identity & money**	`users`, `tokenLedger`, `payments`; register/login/me; `applyTokenDelta` with role reconciliation; the header balance pill; `/credits` §23.1 sections 1, 2, 6.	An account registers with 50 🪙, a manual credit promotes to moderator, and the ledger reconstructs the balance exactly.
3	**Content skeleton**	`repos`, `units`, `lessons`, `slideTools`; CRUD; the Repos and Slides shelves; repository detail.	A repo can be built by hand, with units and lessons, and browsed.
4	**The AI layer**	Provider adapters + resolution + rotation; `estimateCost`; the deck prompt (Ch. 10); repair + validation (Ch. 12); `generate.slides` with the payment gate; the mock provider.	A deck generates against a real provider, charges correctly, and a forced failure refunds.
5	**The player**	All nine component renderers; all five evaluation kinds; AI grading; `runs.complete`; lesson logs; **course memory**.	Lesson 2 generated after playing lesson 1 provably does not re-teach lesson 1's material.
6	**Tickets**	`tickets`, `ticketRequests`; issue / spend / transfer / request; the ticket branch of the payment gate; `/credits` sections 4 and 5.	Two concurrent generations cannot double-spend one ticket.
7	**Social**	`posts`, favourites, follows, assignments; Feed; Gallery; Users; the language filter and stickers.	Visibility is enforced in SQL and a limit of 30 returns 30 visible rows.
8	**Marketing & Card**	Storyboard, mathboard, image generation, cast, music; the card model, the shared layout function, PNG/PDF/print-sheet export, versions.	Preview and export are pixel-identical, and every download is named from the site slug.
9	**Admin**	Dashboard, users, moderators, payments, flags, settings, finance with live model pricing and budgets, the template catalogue editor.	An admin can set a price and see it change a quote immediately.
10	**Polish**	The Chapter 29 fixes; empty/error/loading everywhere; keyboard pass; contrast audit; translation coverage; the admin annotation overlay.	Chapter 31 passes.

Acceptance criteria

ECONOMY

- Summing `tokenLedger.delta` for any user equals their `tokenBalance`. No exceptions.
- No AI code path reaches a provider without a prior quote and debit — verified by grep and by review.
- A provider failure leaves the balance exactly as it was.
- Crediting a plain user promotes them to moderator; draining them demotes and un-verifies. An admin is unaffected in both directions.
- Two concurrent generations with one ticket on hand: one succeeds on the ticket, the other pays coins or is refused. Never both on the ticket.

GENERATION

- Every slide in every generated deck carries at least one real explanatory paragraph.
- Every slide conforms to a layout from the offered catalogue.
- Correct MCQ answers are distributed across positions, not clustered at index 0.
- A deck generated after prior lessons does not re-define any concept in the course memory.
- A commercial deck contains no evaluations; a walkthrough and a news deck likewise.
- Navigating away mid-generation still results in a saved deck.

VISIBILITY

- A private post returns `NOT_FOUND`, not `FORBIDDEN`, to a stranger.
- A list with `limit: 30` returns up to 30 rows the viewer can see.
- Content assigned to you is never hidden by the language filter.
- The Spanish filter shows only Spanish; the English filter shows everything.

INTERFACE

- Every async surface has three visually distinct states, and a failure never renders as empty.
- Every charged control shows its price before it is pressed.
- Full keyboard traversal with a visible focus ring throughout.
- Body text meets 7:1 contrast in both themes, including over every glass surface.
- No layout shift when skeletons resolve.
- Zero untranslated user-facing strings at ship; zero dead dictionary keys.

CORRECTNESS

- The typecheck is the project's real build (`tsc -b` with project references), not a `--noEmit` that silently checks nothing. *Confirm this on day one* — a root `tsconfig.json` with `"files": []` makes `tsc --noEmit` a no-op that will sail past syntax errors.
- Boot migrations are idempotent; running them twice changes nothing.
- A request arriving mid-migration either succeeds or returns a real error — never an empty list.

PART IX · CHAPTER 32

Environment and deployment

32.1 Variables

Variable	Required	Notes
`DATABASE_URL`	Yes	Postgres connection string.
`APP_SECRET`	Yes in prod	JWT signing.
`NODE_ENV`	—	—
`PORT`	—	Defaults to 3000.
Provider keys	No	Optional per capability; users may bring their own, and platform keys can live in settings instead.
`*_ALLOW_MOCK_AI`	No	1 enables the offline generator. Dev and CI only.

NEVER THROW AT IMPORT TIME

Reading env vars must warn, not throw. A throw at module scope crashes the whole serverless function and every request returns an opaque 500 with no clue which variable is missing. Warn loudly, and let a genuinely missing `DATABASE_URL` surface as a normal JSON database error the client can display.

32.2 Build and health

```
npm run dev       vite dev server + API
npm run check     tsc -b            ← the real typecheck
npm run lint      eslint .
npm test          vitest run
npm run build     vite build && esbuild api/boot.ts --bundle --platform=node --format=esm
npm start         node dist/boot.js
```

Ship a `health` endpoint that reports, without leaking the connection string: whether `DATABASE_URL` is visible to the running function, whether the database answers, the host, the database name, the SSL mode, and the Node version. Every deployment problem this system had was diagnosable from those six facts.

32.3 Seeds

Seed one admin, one moderator and one plain user with known passwords, a handful of repos across at least three templates, one generated deck, one completed run (so course memory has something in it), and one published post. A fresh clone should be explorable immediately — a blank database makes every surface look broken.

END OF SPECIFICATION

Parts II–VI are the contract: build them and the product works. Parts VII–VIII are the brief: build them and it is worth using. If the two ever conflict, the behaviour spec wins and the design adapts — except on the two invariants, which never bend: **everything the AI makes is paid for out of the acting user's credits**, and **a failed generation is never charged**.